

Article

Addressing Data Scarcity in Malware Classification via Pixel-Level Synthetic Image Generation

Mounika Krishna Teja Karumudi and Fabio Di Troia * 

Department of Computer Science, San Jose State University, San Jose, CA 95192, USA;
mounika.karumudi@gmail.com

* Correspondence: fabio.ditroia@sjsu.edu

Abstract

Deep learning-based malware classification using image representations has emerged as a highly effective paradigm for threat detection. However, training robust neural networks is frequently bottlenecked by data scarcity and severe class imbalances in real-world repositories. This study investigates the viability of using an autoregressive PixelCNN framework to synthesize high-fidelity, class-specific malware images to augment limited training distributions. Utilizing the benchmark Maling dataset, we systematically evaluate a Convolutional Neural Network (CNN) classifier across varying ratios of synthetic-to-authentic data under strict data scarcity constraints (ranging from 10 to 80 authentic samples per family). Our experimental results reveal that while PixelCNN successfully replicates intricate, byte-level micro-textures, classifiers trained exclusively on synthetic data experience catastrophic performance degradation, yielding an accuracy of just 3%. Crucially, however, the introduction of a minimal authentic data anchor (15% to 20%) restores functional decision boundaries, immediately elevating classification accuracy up to 72%. Furthermore, performance saturates rapidly once the training matrix reaches a 50/50 synthetic-to-authentic split, achieving up to 82% classification accuracy, rendering it highly competitive with the 89% accuracy upper bound of a fully authentic baseline. These findings demonstrate an exceptional degree of data efficiency, proving that generative autoregressive augmentation can halve the authentic data collection burden in cybersecurity workflows provided a minor, real-world baseline anchor is preserved.

Keywords: Convolutional Neural Network; PixelCNN; malware classification; deep learning; synthetic data generation



Academic Editor: Zbigniew Kotulski

Received: 24 May 2026

Revised: 24 June 2026

Accepted: 27 June 2026

Published: 30 June 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Malware remains a pervasive and existential threat to digital ecosystems, with the volume, velocity, and sophistication of attacks reaching unprecedented levels. Recent cybersecurity threat indices highlight an aggressive escalation in threat landscapes; for instance, corporate ransomware velocity and zero-day execution frequencies have surged dramatically, alongside a massive influx of millions of previously unrecorded, highly evasive polymorphic variants targeting enterprise infrastructures annually [1,2]. As these complex threats evolve across multi-architecture environments, manual reverse-engineering and traditional analysis of malicious binaries become entirely impractical. This creates an urgent and critical demand for automated, highly scalable classification techniques that remain resilient against rapid structural mutations and intentional code obfuscation [3,4].

Historically, malware detection has relied on two primary paradigms: signature-based detection and machine learning (ML) approaches [5], including deep learning (DL) architectures [6]. Inspired by the landmark breakthroughs of Convolutional Neural Networks (CNNs) in computer vision, researchers have successfully adapted these models for cybersecurity by transforming raw executable binary byte sequences into 2D grayscale images. These image-based approaches frequently outperform traditional static and dynamic analysis in both macroscopic classification accuracy and computational throughput, as they effectively leverage the spatial correlation and structural topology inherent to compiled code arrays without requiring code execution or unpacking [7,8].

The underlying efficacy of these visual classification frameworks is rooted in the fact that malware families often exhibit highly distinct texture patterns and structural symmetries due to code reuse, shared libraries, and modular development paradigms [9]. As illustrated in Figure 1, instances within a single family (e.g., FakeRean) maintain an exceptionally high degree of visual consistency, whereas samples originating from distinct families exhibit marked macroscopic structural deviations (see Figure 2). However, while standard deep CNNs excel at capturing global, macro-level structural features, they often fail to perceive localized, pixel-level cross-dependencies. This limitation becomes highly problematic when analyzing sophisticated, modern variants that utilize packing or localized encryption techniques specifically designed to smooth out global visual feature representations [10].

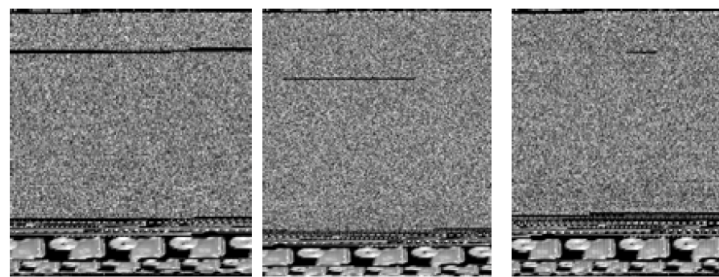


Figure 1. Visual representations of three malware samples from the FakeRean family.

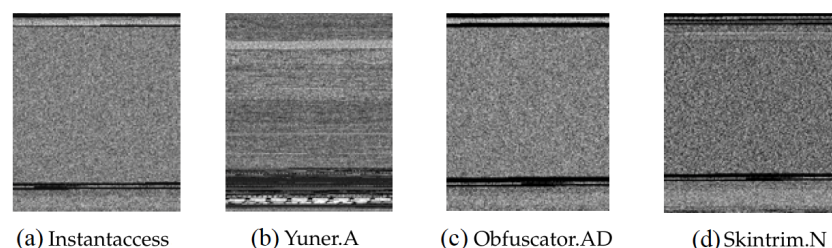


Figure 2. Visual representations from various malware families demonstrating distinct structural textures.

To address these fundamental structural limitations, and to mitigate the pervasive, real-world challenge of severe class imbalances and limited labeled samples in threat repositories, this work explores the utility of the Pixel Convolutional Neural Network (PixelCNN). Unlike traditional adversarial generative architectures or standard feedforward CNNs, PixelCNN operates as a strict autoregressive model that analyzes and synthesizes images one pixel at a time, conditioning each step explicitly on all previously generated pixel values [11]. This architectural property allows the model to map the fine-grained joint probability distribution and replicate the intricate, byte-level “micro-textures” of functional malware code. Consequently, this research systematically evaluates PixelCNN’s dual-operational capability: its performance as a robust feature classifier under strict data

constraints, and its capacity to synthesize high-fidelity, class-specific malware samples to aggressively augment training data distributions.

Utilizing the foundational, highly benchmarked Maling dataset, which comprises 9339 samples across 25 distinct threat families [9], this paper comprehensively demonstrates how generative autoregressive augmentation can structurally reinforce downstream classification resilience. We specifically investigate a critical frontier in generative cybersecurity workflows: whether synthetic samples can successfully replicate authentic distribution boundaries to defend classifiers against catastrophic performance degradation, thereby determining the minimum real-world anchor required to train highly accurate networks under severe data scarcity.

The remainder of this paper is organized as follows: Section 2 reviews the state of the art in generative modeling and image-based malware analysis. Section 3 details the mathematical mechanics of the PixelCNN architecture and our proposed augmentation framework. Section 4 presents our experimental setup, evaluation metrics, and empirical results, followed by concluding remarks and future research vectors in Section 5.

2. Related Work

In response to the growing challenge of malware identification and classification, researchers have increasingly explored image analysis combined with machine learning to design more effective detection systems [12]. This section reviews key studies that have shaped the development of image-based malware classification techniques and identifies current research gaps that this work aims to address.

One of the earliest contributions in this area was presented by Nataraj et al. [9], who introduced a framework for converting malware binaries into grayscale images. Rather than analyzing raw binary sequences, the authors extracted GIST texture features from these images and classified them using the k -nearest neighbors (k -NN) algorithm. Their model successfully distinguished among 25 malware families, establishing an early foundation for image-based malware analysis. Building on this work, Yajamanam et al. [13] applied convolutional neural networks (CNNs) to the same dataset, achieving even higher performance in malware detection. Similarly, in a more recent study [14], the authors employed a comparable deep learning approach, further reinforcing the view that automated feature learning substantially improves accuracy over handcrafted representations. Furthermore, researchers have expanded these mapping techniques, transforming executable binaries into grayscale, plasma colormaps, and structural Portable Executable (PE) file entropy layouts, to optimize how neural networks perceive varying structural threat patterns across diverse hardware platforms [15].

The field has recently shifted toward generative modeling to overcome the scarcity of labeled malware data. Generative Adversarial Networks (GANs) have been widely adopted for this purpose. For instance, MalGAN was proposed to generate adversarial malware examples that can bypass detection systems while maintaining functional integrity [16]. Similarly, researchers have utilized GAN-based architectures to augment training sets, improving the resilience of classifiers against previously unseen variants [17]. To combat severe dataset imbalances across rare malware families, conditional variants and Deep Convolutional GANs (DCGANs) have been deployed to equalize class distributions prior to classifier training. However, these systems still struggle with complex structural transformations in evolving threat environments, prompting hybrid attempts that model grayscale image distributions in rigid, sequential, pixel-by-pixel patterns to preserve critical functional and structural boundaries [18].

More recently, Denoising Diffusion Probabilistic Models (DDPMs), or Diffusion models, have emerged as a powerful alternative for synthetic data generation. Unlike GANs,

which can suffer from training instability and mode collapse, Diffusion models learn to reverse a gradual noise process to generate high-fidelity images [19]. Recent studies have begun exploring these models for cybersecurity applications, noting their ability to produce more diverse and structurally complex samples compared to traditional generative frameworks [20].

However, despite the success of GANs and Diffusion models, a specific challenge persists: capturing the sequential, pixel-level dependencies that define the “micro-textures” of malware code. While GANs focus on global distribution and Diffusion models on iterative refinement, they can sometimes overlook the fine-grained structural nuances essential for distinguishing closely related malware families. This limitation highlights the need for autoregressive models capable of high-fidelity, pixel-by-pixel synthesis.

PixelCNN provides a promising direction in this regard. By modeling the distribution of image pixels sequentially, it can capture intricate local dependencies and reproduce detailed visual textures [11]. Crucially, the mathematical stability of maximizing log-likelihood via PixelCNN has already demonstrated immense viability in other highly skewed, unbalanced classification domains, proving highly effective at capturing complex high-dimensional textures where traditional GANs undergo training instability or mode collapse [21].

Beyond traditional convolutional networks, the state of the art in generative modeling has increasingly shifted toward Transformer-based architectures that operate on discrete visual tokens. Pioneered by frameworks like Vector Quantized Variational Autoencoders (VQ-VAE) [22], Image Transformers [23], and subsequent autoregressive synthesis pipelines like VQ-GAN [24], these models treat image generation similarly to sequence-to-sequence language modeling. Instead of predicting raw pixel intensities directly, these modern paradigms compress high-dimensional visuals into a discrete quantized codebook space, leveraging a Transformer decoder to handle long-range structural dependencies. While these token-based vision transformers offer superior global semantic coherence over complex, large-scale datasets, their multi-billion parameter footprints require massive source distributions to avoid severe manifold instability. Consequently, under conditions of extreme data starvation, the localized, lean autoregressive modeling properties of PixelCNN remain uniquely advantageous for capturing the rigid, translation-invariant byte blocks characteristic of binary malware imagery.

However, a persistent vulnerability in generative data augmentation frameworks is the potential divergence between visual fidelity and downstream classification utility. Prior benchmarks in computer vision have demonstrated that synthetic samples minimizing standard distribution distance metrics can still induce catastrophic performance degradation when utilized to train deep classifiers in complete isolation [25]. This vulnerability underscores a critical, under-explored threshold in cybersecurity workflows: determining the exact minimum anchor of authentic data required to structurally stabilize decision boundaries when relying heavily on autoregressive synthesis. This work aims to bridge the gap between generative modeling and classification by evaluating PixelCNN’s ability to produce authentic synthetic data that mirrors genuine malware distributions, thereby defining the optimal equilibrium between synthetic expansion and real-world anchoring even under severe data scarcity constraints.

3. Methodology

This section details the Maling dataset and the architectural components of the PixelCNN framework, including masked convolutional layers, residual blocks, and the 1×1 convolutional classifier. It further describes the autoregressive training strategy employed to generate synthetic malware samples.

3.1. Dataset

This research utilizes the Maling dataset, a benchmark collection comprising 9339 malware samples across 25 distinct families [9,26]. The images in this dataset are generated by reading the raw malware binary files byte-by-byte and mapping each individual byte directly to a grayscale pixel intensity value ranging from 0 (black) to 255 (white). The resulting visual representations organize these sequential byte values into fixed-width matrix structures to highlight the spatial correlations of the code. The dataset is characterized by a significant class imbalance, providing a realistic scenario for evaluating the generative model's ability to augment minority classes. The distribution of samples per family is detailed in Table 1.

Table 1. Maling dataset distribution by malware family and type.

Malware Family	Malware Type	Sample Count
Adialer.C	Dialer	122
Agent.FYI	Trojan	116
Allapple.A	Worm	2949
Allapple.L	Worm	1591
Alueron.gen!J	Trojan	198
Autorun.K	Worm	106
C2LOP.P	Trojan	146
C2LOP.gen!g	Trojan	200
Dialplatform.B	Dialer	177
Dontovo.A	Trojan	162
FakeRean	Rogue	381
Instantaccess	Dialer	431
Lolyda.AA1	Trojan PWS	213
Lolyda.AA2	Trojan PWS	184
Lolyda.AA3	Trojan PWS	123
Lolyda.AT	Trojan PWS	159
Malex.gen!J	Trojan	136
Obfuscator.AD	Trojan	142
Rbot!gen	Trojan	158
Skintrim.N	Trojan	80
Swizzor.gen!E	Trojan	128
Swizzor.gen!I	Trojan	132
VB.AT	Worm	408
Wintrim.BX	Trojan	97
Yuner.A	Worm	800
Total		9339

To mitigate the severe class imbalance inherent to the Maling dataset (where class support ranges from 80 to 2949 samples), a strict class-balanced sampling strategy was enforced during the training phase. Rather than using naive random sampling, mini-batch generation utilized a stratified, weighted sampling mechanism. This approach dynamically adjusted selection probabilities to ensure that every malware family possessed an equal statistical probability of being represented in any given training batch, preventing the downstream CNN from developing an operational bias toward majority classes like *Allapple.A*. Furthermore, the synthetic augmentation pipeline itself acted as an algorithmic balancer; by generating uniform sample counts across all classes within our targeted scarcity baselines (80, 40, 20, and 10 samples), the framework regularized the uneven empirical distribution, stabilized macro metrics, and prevented minority classes from suffering feature suppression.

3.2. PixelCNN Architecture

To address the challenge of dataset scarcity, this study utilizes PixelCNN, a powerful generative model belonging to the family of autoregressive networks [27]. To understand the mechanics of PixelCNN, it is helpful to contrast autoregression with standard linear regression. In traditional multiple regression, a dependent variable y is predicted using a set of distinct, independent variables (e.g., A, B, D) via a linear combination:

$$y = m_1A + m_2B + m_3D + \dots + C \quad (1)$$

where A, B , and D are independent predictor variables, m_1, m_2 , and m_3 are their respective learned coefficients weighting the contribution of each predictor, and C is a constant intercept term.

In contrast, an autoregressive model assumes that the present value of a variable, y_t , is directly dependent upon its own historical sequence:

$$y_t = m_1y_{t-1} + m_2y_{t-2} + m_3y_{t-3} + \dots + C \quad (2)$$

where y_t denotes the value of the variable at the current time step t , y_{t-1} , y_{t-2} , and y_{t-3} denote its values at the preceding time steps, m_1, m_2 , and m_3 are the learned coefficients assigned to each lagged term, and C is a constant intercept.

PixelCNN adapts this sequential forecasting logic to the spatial domain of image generation. By framing an image as a flattened sequence of pixels processed in a row-by-row, pixel-by-pixel fashion, PixelCNN predicts the probability distribution of each new pixel conditioned strictly on the known values of all previously generated pixels. Conceptually, this spatial conditioning mirrors how recurrent architectures process temporal sequences [27]. By explicitly learning the conditional distribution of each pixel relative to its predecessors, PixelCNN captures fine-grained, byte-level micro-textures that standard convolutional architectures often overlook. This complete system framework is illustrated in Figure 3.

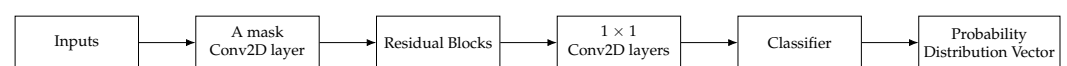


Figure 3. Overview of the PixelCNN architecture, illustrating the flow from masked convolutions through residual blocks to the final softmax classifier.

To maintain the autoregressive property, that is, ensuring that a pixel's prediction depends only on previously observed data, PixelCNN utilizes masked convolutional filters. These filters restrict the receptive field to pixels located above and to the left of the target pixel [11,28]. The model employs two mask types:

- Type A Mask: Applied exclusively to the initial layer to exclude the center pixel, ensuring the network does not “see” the value it is tasked with predicting.
- Type B Mask: Applied to all subsequent layers to allow the center pixel (which now contains intermediate feature information) to contribute to deeper computations.

To facilitate the training of deeper networks, the architecture incorporates residual blocks. These blocks utilize shortcut connections to mitigate vanishing gradient issues and stabilize the learning process [28]. Within each block, a bottleneck structure is used: the channel dimension is first reduced, a masked convolution is applied, and the original dimensionality is restored (see Figure 4).

Following the feature extraction layers, 1×1 convolutions (pointwise convolutions) are employed. These layers act as per-pixel fully connected networks, allowing the model to learn complex cross-channel relationships without violating spatial constraints. The archi-

texture concludes with a softmax classifier, which outputs a discrete probability distribution over the 256 possible intensity values for each pixel.

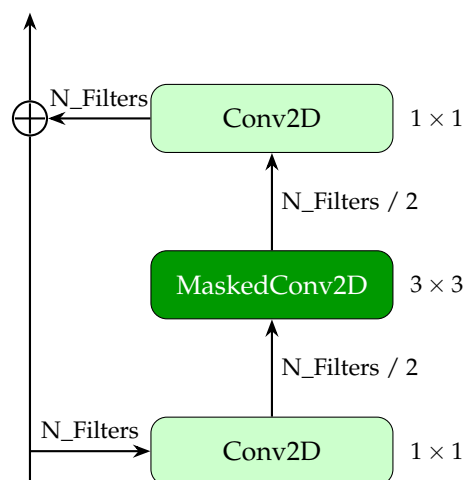


Figure 4. Architecture of the PixelCNN residual block, demonstrating the bottleneck layers and the identity shortcut connection.

3.3. Autoregressive Training and Generation

The training objective is to model the joint distribution of the malware image x as a sequence of conditional probabilities, following a raster-scan order (row-by-row, pixel-by-pixel). The probability of an $n \times n$ image is defined as:

$$p(x) = \prod_{i=1}^{n^2} p(x_i | x_1, \dots, x_{i-1}) \quad (3)$$

where x denotes the full malware image, $n \times n$ is the spatial resolution of the image yielding a total of n^2 pixels, x_i is the intensity value of the i -th pixel in raster-scan order, and $x_1, \dots, x_{i-1}$ represents the set of all previously observed pixels that condition the prediction of x_i .

During the training phase, input pixel values are normalized to the $[0, 1]$ range via $\hat{x}_i = x_i/255$, where $x_i \in \{0, 1, \dots, 255\}$ is the original integer pixel intensity. This scaling ensures that input features lie within a consistent numerical range, promoting stable gradient updates. The network outputs a discrete 256-channel softmax distribution over the possible pixel intensities, and the model is optimized using categorical cross-entropy loss against the original integer targets. Once trained, the generation process is iterative: the model predicts a probability distribution for the first pixel, samples a discrete value, and then uses that value as context for predicting the next pixel. This sequential dependency allows the model to synthesize high-fidelity malware images that maintain the structural logic of the original malware families.

While autoregressive models like PixelCNN possess a theoretical dual capability, namely, generating synthetic imagery and performing direct classification via class conditional likelihood estimation, evaluating PixelCNN as a standalone classifier falls outside the scope of this study. In this framework, PixelCNN is utilized strictly as an upstream generative model tasked with alleviating extreme data scarcity. The downstream classification task is entirely delegated to a dedicated, separate CNN classifier optimized specifically for discriminative feature extraction.

4. Experiments and Results

This section elaborates on the hyperparameter optimization and training phases of the PixelCNN generative model, followed by a comprehensive evaluation of the downstream classification performance when utilizing varying ratios of synthetic and authentic malware images.

4.1. PixelCNN Training and Optimization

The training procedure for the PixelCNN network was conducted in sequential phases to optimize image fidelity and stabilize the loss function. The initial baseline architecture yielded a high training loss of approximately 3.5. To improve convergence, an initial hyperparameter exploration was implemented: the learning rate was reduced from 0.01 to 0.001 to ensure smoother gradient descent, and the batch size was lowered from 128 to 32 to provide more frequent, stable weight updates.

Additionally, adjusting the target color depth yielded a significant performance optimization. While lowering the pixel intensity levels from 32 to 8 brought the loss down to approximately 2.0, further reductions to 4 and 2 levels lowered the loss to 1.28 and 0.637, respectively, but resulted in severe visual degradation and black artifacting. Consequently, a pixel intensity quantization level of 8 was retained to maintain an optimal balance between visual structural fidelity and loss minimization.

Following this initial tuning phase, the final optimized generation quality was achieved by scaling the network capacity to the following configuration: an image size of 64×64 pixels, 8 pixel quantization levels, 256 filter channels, 7 residual blocks, and a micro-batch size of 2. Under this specialized high-capacity regime, the cross-entropy loss started at 1.845 in Epoch 1 and successfully converged to a stable floor of 1.422 by Epoch 366 (see Figure 5).

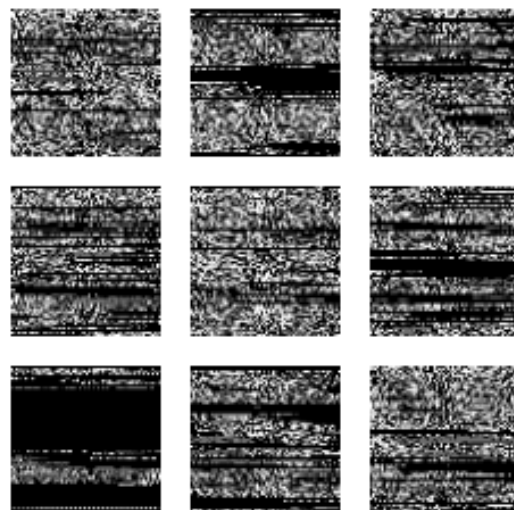


Figure 5. Sample generation result at Epoch 366.

To ground these selection choices and ensure reproducibility, the broader hyperparameter exploration bounds and structural grid search parameters examined prior to final model locking are documented in Table 2. This search space was systematically evaluated against a dedicated 15% validation split of the training data.

During the classification phase, convergence trajectories for the downstream CNN classifier were monitored via validation loss curves, with training terminating once the categorical cross-entropy loss hit a stable asymptotic floor (typically by Epoch 60) to protect the model from overfitting to the scarce training assets.

Table 2. Hyperparameter Grid Search Space Boundaries and Configuration Parameters.

Model Pipeline/Parameter	Search Space Range	Explored Baseline Value
Generative PixelCNN		
Masked Convolutional Layers	[4, 6, 8, 12]	6 Layers
Filter Dimension (Kernel Size)	[3 × 3, 5 × 5, 7 × 7]	3 × 3
Feature Map Channels	[32, 64, 128, 256]	64 Channels
Learning Rate (Adam)	[10 ^{−2} , 10 ^{−3} , 10 ^{−4}]	10 ^{−3}
Batch Size	[16, 32, 64]	32
Downstream CNN Classifier		
Convolutional Blocks	[2, 3, 4, 5]	3 Blocks
Initial Learning Rate	[10 ^{−3} , 5 × 10 ^{−4} , 10 ^{−4}]	10 ^{−3}
Dropout Rate	[0.2, 0.3, 0.4, 0.5]	0.3
Batch Size	[16, 32, 64]	32
Optimizer	[SGD, Adam, RMSprop]	Adam

4.2. Class-Specific Image Generation

To resolve early issues where generalized generation failed to capture specific structural families, the PixelCNN model was transitioned to class-specific training. Utilizing an NVIDIA A100 GPU, separate models were trained for each of the 25 malware families. The input resolution was also increased to 128 × 128, resulting in highly distinct, class-specific synthetic malware samples (see Figure 6).

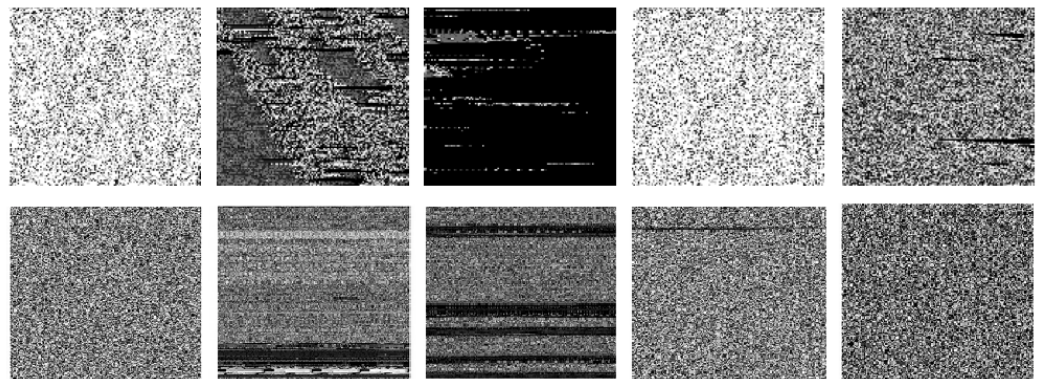


Figure 6. Class-specific image generation results across distinct malware families. The top row shows synthetic samples, while the bottom row shows real samples.

4.3. Impact of Source Data Volume on Generator Quality

Before evaluating mixed datasets, we first assessed how the volume of initial authentic data impacts PixelCNN's generation quality. Separate PixelCNN models were trained using 80, 40, 20, and 10 authentic samples per family. In all conditions, the held-out real images used for final evaluation were drawn from the same fixed test partition, defined prior to any training, ensuring a consistent and comparable evaluation benchmark across all source data volumes. Each model was then used to generate a uniform baseline of 100 synthetic samples per family. Subsequent classification models were trained entirely on these synthetic samples and evaluated against the remaining held-out real images. The classification accuracy decayed severely as the PixelCNN source data was reduced: models trained on the 80-sample PixelCNN generation achieved 28% accuracy, the 40-sample generation achieved 4%, and both the 20- and 10-sample generations collapsed to 3%. Figure 7 visually confirms that PixelCNN's ability to generate useful, generalizing data is highly dependent on the size of its initial authentic training set.

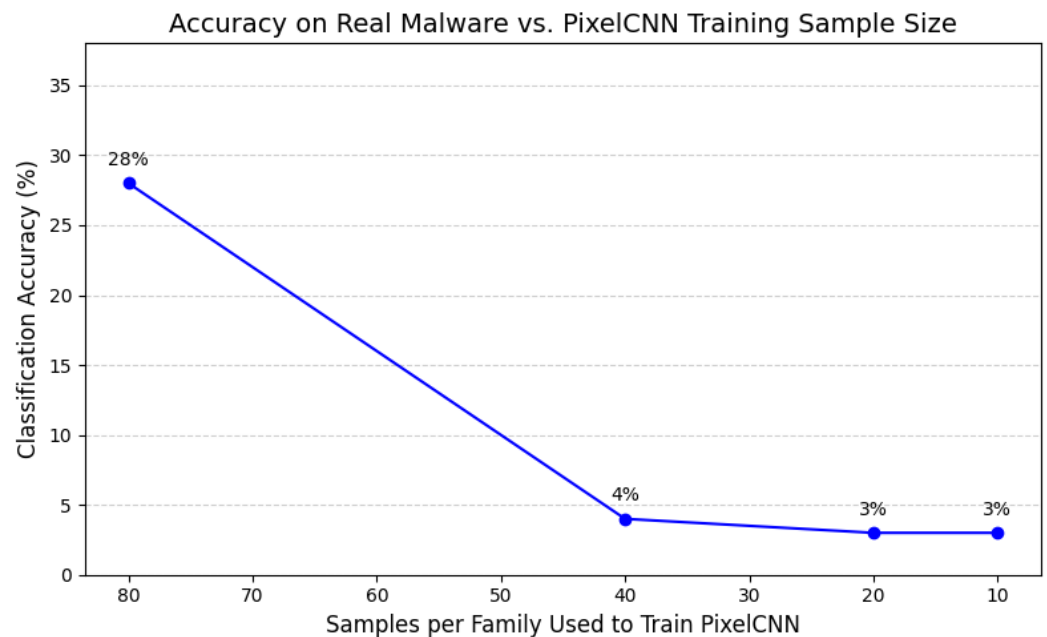


Figure 7. Accuracy decay on real malware correlated with reduced PixelCNN training sample size.

4.4. Classification Performance with Synthetic Augmentation

With high-fidelity, class-specific synthetic data available, a rigorous experimental framework was designed to test the viability of supplementing authentic training data with synthetic data.

Four primary training baselines were established based on the number of available images per family: 80, 40, 20, and 10. Across each baseline, a series of experiments progressively shifted the ratio of the training data from 100% Synthetic to 100% Authentic. All models were subsequently evaluated against the designated test split of real malware images.

4.5. Computational Efficiency and Hardware Environment

To evaluate the practical deployment feasibility of the framework, the training and generation durations were logged under a high-performance cloud infrastructure profile. All computational pipelines were executed within a Google Colab environment backed by an enterprise-grade NVIDIA A100 Tensor Core GPU (equipped with up to 40 GB/80 GB of high-bandwidth VRAM) and a high-throughput multi-core cloud CPU allocation.

The computational overhead for both processing phases is detailed as follows:

- **Model Training Duration:** Capitalizing on the tensor acceleration of the A100 architecture, the upstream generative PixelCNN requires approximately 1.5 to 2.5 s per epoch when optimizing on the restricted data baseline under the final high-capacity configuration (256 channels, 7 residual blocks). Consequently, the complete 366-epoch optimization run finishes execution in roughly 10 to 15 min. The downstream CNN classifier converges almost instantly, completing its training phase in under 60 s.
- **Sample Generation Throughput:** Due to the sequential, pixel-by-pixel sampling constraints inherent to autoregressive architectures, image generation scales linearly with spatial resolution. For a target resolution of 64×64 pixels under 8 quantization levels, the runtime generation cost is approximately 0.05 to 0.10 s per fully synthesized malware sample on the A100. Producing a complete synthetic reinforcement batch of 150 variant images requires less than 15 s of continuous execution.

These benchmarks demonstrate that by utilizing modern cloud hardware acceleration, the framework completely mitigates the traditional computational bottlenecks of

autoregressive generation, proving its high practical viability for real-time automated data augmentation workflows.

4.6. Experimental Results Summary

The global classification accuracies across all baselines and split ratios are consolidated in Table 3.

Table 3. Classification Accuracy Across Varying Baselines and Synthetic/Real Ratios.

Gen %	Real %	80 Samples/Class	40 Samples/Class	20 Samples/Class	10 Samples/Class
100%	0%	28%	4%	3%	3%
90%	10%	31%	5%	3%	3%
85%	15%	61%	61%	61%	-
80%	20%	72%	71%	71%	69%
70%	30%	78%	78%	77%	76%
60%	40%	80%	81%	77%	76%
50%	50%	82%	82%	80%	81%
40%	60%	83%	82%	83%	82%
30%	70%	81%	84%	83%	82%
20%	80%	84%	84%	82%	83%
10%	90%	89%	88%	87%	85%
0%	100%	89%	89%	88%	86%

The most notable observation across all four baselines is the failure of the models when trained exclusively or nearly exclusively on synthetic data (0% to 10% real data). Despite PixelCNN’s ability to replicate the visual “texture” of malware, the classifiers require a minimum of 15% to 20% real data to achieve a functional baseline (ranging from 61% to 72%). This suggests that while synthetic images provide excellent spatial coverage, authentic samples act as the critical “ground truth” anchors required for the CNN to learn specific decision boundaries between similar families.

Minor non-monotonic fluctuations are observable across specific evaluation vectors in Table 3 (for instance, the isolated performance drop at the 30% Gen/70% Real split ratio within the 80-sample baseline). These localized variances are attributed to a combination of mini-batch stochastic optimization noise and data subset selection bias, which are naturally pronounced under extreme data starvation constraints. Because the authentic baseline sets are restricted to tiny seed counts (e.g., 10 to 80 samples), minor differences in the underlying structural diversity of the randomized cross-validation folds can cause small shifts in the downstream classifier’s gradient paths during the final training epochs. Rather than indicating a breakdown in the overarching generative scaling trends, these subtle micro-fluctuations reflect standard convergence variances inherent to deep network optimization over highly restricted data distributions.

As shown in Table 3 and visualized in Figure 8, performance begins to saturate as the ratio approaches 50% real data. For example, in the 80-sample baseline, the accuracy gain between 50% real data (82%) and 100% real data (89%) is only 7%. This demonstrates a high degree of data efficiency: by utilizing PixelCNN augmentation, comparable accuracy can be achieved while significantly reducing the burden of authentic data collection.

The detailed classification reports for all experiments (see Appendix A) reveal that certain families, such as Allapple.A and Allapple.L, were successfully identified even with high synthetic ratios. This is likely due to their distinct, repetitive structural patterns which PixelCNN models with high fidelity. Conversely, families like Swizzor.gen!E and Swizzor.gen!I remained difficult to classify until the real-world data ratio increased, indicating that these families possess subtle, non-repetitive features that are much harder to synthesize accurately.

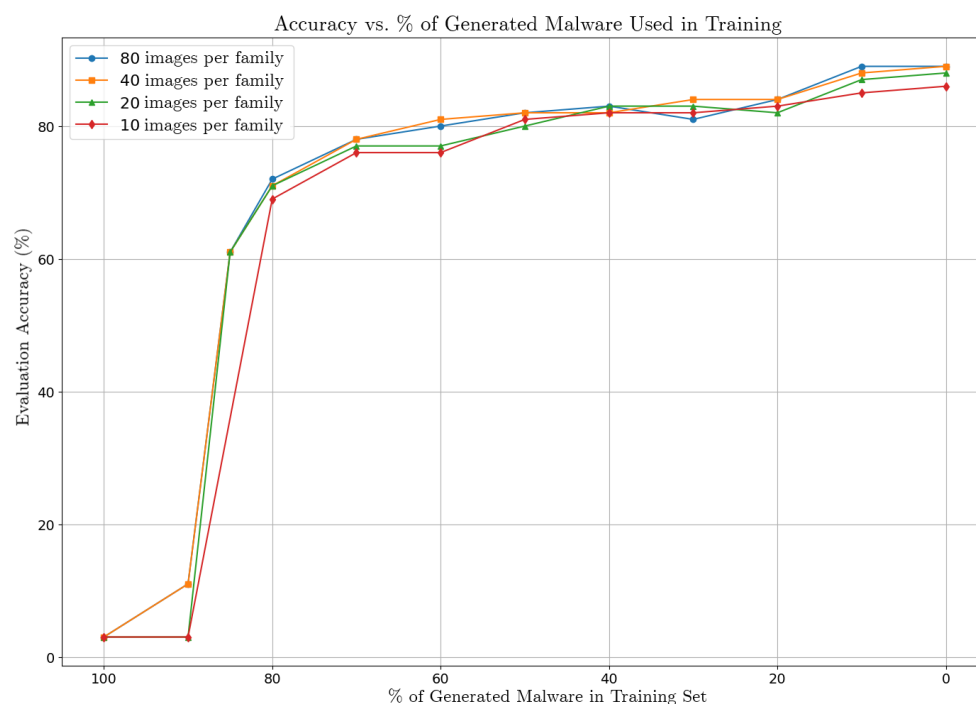


Figure 8. Accuracy vs. Percentage of Generated Malware Used in Training across all baselines.

Figure 9 illustrates the model's prediction confidence evolution at three key stages of the synthetic-to-real ratio.

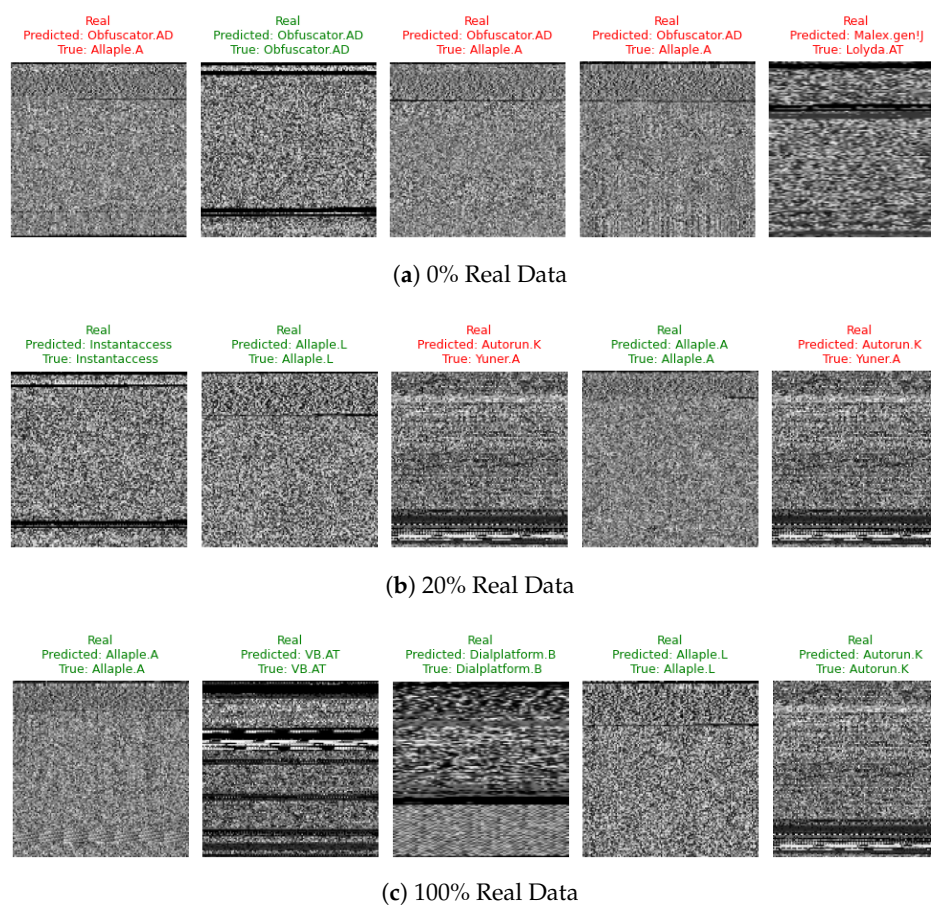


Figure 9. Visualizing prediction confidence evolution. High-intensity areas represent strong class identification.

Detailed classification reports for the splits with at least 30% real data to 90% real data, alongside their corresponding visualization figures, are included in Appendix A to provide a granular view of performance fluctuations across minority classes.

4.7. Comparative Analysis Against Generative Baseline Modalities

To fully evaluate the comparative utility of the proposed PixelCNN data augmentation matrix under extreme data constraints, we contextualize our performance against both a representative GAN-based framework [17] and a state-of-the-art Denoising Diffusion pipeline [20] found in recent malware literature.

In the adversarial domain, Trehan and Di Troia [17] utilized Wasserstein Generative Adversarial Networks with Gradient Penalty (WGAN-GP) to synthesize 1D sequential opcode embeddings, reporting multi-class family classification accuracy ranging from 70.0% to 82.0% across major families like *Zbot* using Random Forest engines. In the diffusion domain, Bao et al. [20] leveraged Natural Language Processing (NLP) tokenization combined with a modified generative Diffusion model to augment minority threat classes, ultimately achieving a downstream classification accuracy of 96.0%. Both baseline frameworks operate on 1D sequential feature spaces (tokenized opcode sequences and structural embeddings), whereas our architecture synthesizes 2D spatial byte matrices evaluated directly via a deep Convolutional Neural Network (CNN). As a result, a direct algorithmic replication on identical data splits is unfeasible. Instead, our work isolates downstream evaluation metrics across progressive synthetic-to-authentic blending ratios under identical class-scarcity constraints (10, 20, 40, and 80 samples per class).

The downstream evaluation engine utilized to validate our generated data matrices is a deep convolutional architecture consisting of progressive visual feature extraction blocks—Conv2D(64, 3x3) → MaxPooling2D → Conv2D(128, 3x3) → MaxPooling2D → Conv2D(256, 3x3) → GlobalAveragePooling2D—terminating in a Dense softmax layer configured to the 25 target threat families.

When evaluating the data metrics in Table 3 against these broader generative trends, several critical structural insights emerge. Traditional adversarial pipelines often undergo severe mode collapse or require extensive text corpora to achieve stability. Meanwhile, diffusion-based frameworks (e.g., Bao et al. [20]) provide exceptional global distribution alignment but rely heavily on complex NLP pre-processing pipelines to extract semantic context before generation.

Conversely, our pure synthetic autoregressive pipeline (100% Synthetic/0% Real) exhibits a complete failure of downstream task utility, collapsing to an accuracy of 3.0% under extreme scarcity constraints (10 to 20 samples). This empirical reality proves a key structural distinction: while autoregressive processes minimize distribution log-likelihood effectively to reproduce hyper-realistic, localized visual micro-textures, pure synthetic generations lack the global structural constraints required to establish independent downstream decision boundaries.

Crucially, however, the strategic introduction of a minor authentic data anchor completely resolves this baseline degradation. By anchoring the training distribution with a small fraction of authentic samples (the 80% Synthetic/20% Real split), classification performance immediately climbs to 69% at an extreme floor of just 10 authentic samples per class, stabilizing at 72% as baseline data availability scales. Once a balanced 50/50 synthetic-to-authentic data split is achieved, our framework reaches an accuracy of 81.0% to 82.0% across all scarcity layers. This performance successfully matches the operational utility of more complex, tokenized sequence-generation models in the literature without requiring intensive NLP code parsing or feature extraction pipelines. These findings demonstrate that while pure autoregressive synthesis cannot replace genuine datasets in

isolation, utilizing a blended data matrix presents a highly data-efficient alternative that effectively halves the real-world sample collection burden in cybersecurity workflows.

4.8. Summary and Discussion of Limitations

This study investigated the viability of utilizing PixelCNN-generated synthetic malware images to mitigate data scarcity in deep learning-based malware classification. By systematically evaluating a Convolutional Neural Network (CNN) trained on varying ratios of synthetic and authentic data across severe scarcity baselines (80, 40, 20, and 10 samples per class), we established clear boundaries for the efficacy of synthetic augmentation.

Our empirical findings demonstrate that while PixelCNN excels at replicating the spatial and textural patterns of malware, particularly for highly repetitive families like Allapple, synthetic data cannot operate in isolation. Models trained exclusively on synthetic samples experienced catastrophic performance degradation, yielding a baseline accuracy of merely 3%. However, the introduction of a minimal authentic dataset (15% to 20%) acting as “ground truth anchors” successfully catalyzed the classifier’s ability to learn accurate decision boundaries, immediately elevating accuracy to functional levels (up to 72%). Most significantly, the results prove a high degree of data efficiency; by utilizing synthetic augmentation, we achieved up to 82% accuracy using only half of the target authentic samples, compared to an 89% accuracy ceiling when utilizing a fully authentic dataset. This confirms that generative augmentation can reduce the real-world collection burden by up to 50% while maintaining near-ceiling performance.

Despite these clear data efficiencies, several core algorithmic and structural limitations merit acknowledgment:

- **Class Overlap and Structural Convergence:** Pure image-based feature extraction faces an absolute upper bound when distinguishing between structurally convergent variants. As observed in the *Autorun.K* family, extensive modular code reuse and common packing techniques create identical visual micro-textures in high-dimensional space. As a result, PixelCNN faithfully replicates these overlaps, compounding pre-existing distribution ambiguities.
- **Computational Scalability and Model Footprint:** Training isolated, family-specific models yields an $O(M)$ scaling footprint that presents a deployment bottleneck for enterprise repositories with thousands of classes. To mitigate this, future iterations can transition to a unified *Conditional PixelCNN* using discrete class conditioning vectors, or a vector-quantized latent space system (VQ-VAE/VQ-GAN) to compress textures into a shared codebook, reducing the required footprint to $O(1)$.
- **Generator Overfitting under Data Starvation:** Training individual models on extremely small seed sets (10 to 20 samples) introduces severe overfitting risks. Rather than capturing a generalizable family distribution, the generator memorizes and amplifies idiosyncratic noise or artifacts present in the scarce source data.
- **Architectural and Evaluation Trade-offs:** While modern Diffusion models (DDPMs) offer high semantic fidelity, they suffer from extreme manifold instability and blurred outputs under severe data scarcity, making PixelCNN a more stable baseline for rigid pixel layouts. Additionally, standard generative evaluation metrics like FID are blind to unique binary byte layouts, validating our reliance on downstream classification accuracy (TSTR) as a functional fidelity metric.
- **Feature-Space Representations and Domain Adaptation:** The collapse of purely synthetic training stems from severe covariate shift, as the downstream classifier optimizes entirely around autoregressive generation artifacts rather than actual threat dynamics. Through the lens of semi-supervised domain adaptation, introducing a 15–20% authentic anchor provides the vital supervising signals necessary to execute mani-

fold alignment, constraining the hidden layers to map both domains into a shared, invariant embedding subspace.

5. Conclusions and Future Work

This work demonstrates that while fully synthetic image distributions fail to sustain deep learning malware classifiers independently, combining generative data with a minimal anchor of real-world samples significantly mitigates data scarcity. Our evaluation confirms that generative augmentation can reduce the collection burden of authentic samples by up to 50% while maintaining highly functional classification rates, yielding a practically significant reduction in labeling and acquisition costs for security researchers.

Several avenues remain open for future exploration to enhance this framework. As autoregressive pixel-level generation struggles with complex, non-repetitive structural variations, future research should evaluate modern architectures like Denoising Diffusion Probabilistic Models (DDPMs) or advanced GANs. Additionally, implementing an adaptive blending framework could optimize ratios dynamically based on structural class complexity. Finally, future studies should investigate combining visual structural data with sequential features, such as opcode n -grams, and evaluate if classifiers trained on heavily augmented datasets exhibit heightened vulnerability to adversarial evasion techniques.

Author Contributions: Conceptualization, F.D.T.; methodology, F.D.T. and M.K.T.K.; software, M.K.T.K.; validation, F.D.T.; formal analysis, M.K.T.K.; investigation, M.K.T.K.; resources, M.K.T.K.; data curation, M.K.T.K.; writing—original draft preparation, M.K.T.K.; writing—review and editing, F.D.T.; visualization, F.D.T. and M.K.T.K.; supervision, F.D.T.; project administration, F.D.T. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The dataset used in this article can be found at <https://drive.google.com/drive/folders/1R9t5LrFjp7dU8MocnkCCPzUq7fyscFFX?usp=sharing> (accessed on 26 June 2026).

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Extended Experimental Data

The following subsections provide the full classification reports and model prediction visualizations for the intermediate data configurations, specifically spanning ratios from 70% Synthetic/30% Real to 10% Synthetic/90% Real. These supplemental details are included to demonstrate the incremental performance fluctuations and class-specific optimization trends across all 25 malware families as the proportion of authentic training data progressively increases. Note that these experiments were conducted under conditions of extreme data scarcity, wherein a baseline constraint of only 10 total samples per class was utilized for model training. A comprehensive, in-depth analysis of these class-specific breakthroughs and systemic bottlenecks is discussed in Appendix A.8.

Appendix A.1. Results: 70% Synthetic/30% Real

In this experiment, the model accuracy reached 76%. While many families showed high precision, the model began to struggle with minority classes such as Obfuscator.AD and Rbot!gen.

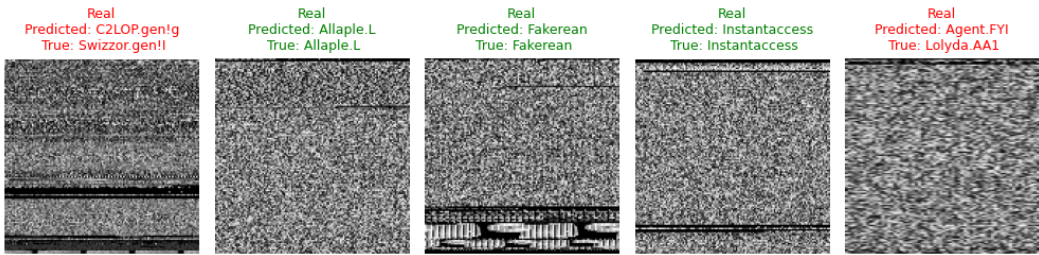


Figure A1. Visualizing Model Predictions: 70% Synthetic/30% Real.

Table A1. Classification Report: 70% Synthetic/30% Real.

Family	Precision	Recall	F1-Score	Support
Adialer.C	0.93	1.00	0.96	122
Agent.FYI	0.90	0.98	0.94	116
Allaple.A	0.99	0.98	0.98	2949
Allaple.L	1.00	0.99	0.99	1591
Alueron.gen!J	0.61	0.91	0.73	198
Autorun.K	0.12	1.00	0.21	106
C2LOP.P	0.30	0.83	0.44	146
C2LOP.gen!g	0.28	0.83	0.42	200
Dialplatform.B	0.95	1.00	0.97	177
Dontovo.A	0.93	1.00	0.96	162
Fakerean	0.80	0.99	0.89	381
Instantaccess	0.74	1.00	0.85	431
Lolyda.AA1	0.96	0.94	0.95	213
Lolyda.AA2	0.90	0.70	0.79	184
Lolyda.AA3	0.72	0.98	0.83	123
Lolyda.AT	0.88	0.99	0.93	159
Malex.gen!J	0.80	0.64	0.71	136
Obfuscator.AD	0.00	0.00	0.00	142
Rbot!gen	1.00	0.02	0.04	158
Skintrim.N	0.00	0.00	0.00	80
Swizzor.gen!E	0.00	0.00	0.00	128
Swizzor.gen!I	0.00	0.00	0.00	132
VB.AT	0.00	0.00	0.00	408
Wintrim.BX	0.00	0.00	0.00	97
Yuner.A	0.00	0.00	0.00	800
Accuracy			0.76	
Macro Avg	0.55	0.63	0.54	9339
Weighted Avg	0.72	0.76	0.72	9339

Appendix A.2. Results: 60% Synthetic/40% Real

In this experiment, the model accuracy remained at 76%. However, we observe an improvement in the F1-scores for several families, including Alueron.gen!J and Obfuscator.AD.

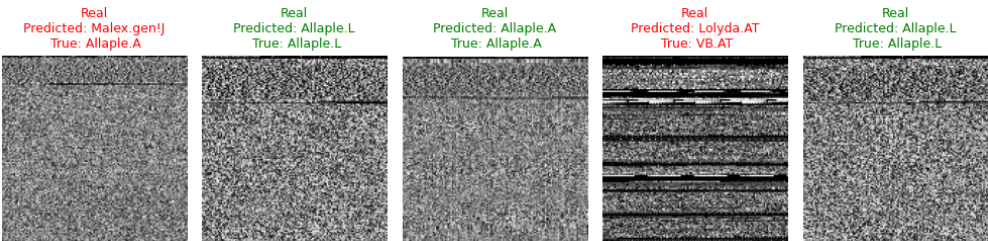


Figure A2. Visualizing Model Predictions: 60% Synthetic/40% Real.

Table A2. Classification Report: 60% Synthetic/40% Real.

Family	Precision	Recall	F1-Score	Support
Adialer.C	0.95	1.00	0.97	122
Agent.FYI	0.99	1.00	1.00	116
Allaple.A	1.00	0.84	0.91	2949
Allaple.L	0.99	1.00	1.00	1591
Alueron.gen!J	0.97	0.98	0.98	198
Autorun.K	0.12	1.00	0.21	106
C2LOP.P	0.30	0.90	0.45	146
C2LOP.gen!g	0.44	0.76	0.56	200
Dialplatform.B	0.53	1.00	0.69	177
Dontovo.A	0.96	0.81	0.88	162
Fakerean	0.99	0.99	0.99	381
Instantaccess	0.99	1.00	0.99	431
Lolyda.AA1	0.97	1.00	0.98	213
Lolyda.AA2	0.71	0.97	0.82	184
Lolyda.AA3	0.98	0.99	0.98	123
Lolyda.AT	0.54	0.99	0.70	159
Malex.gen!J	0.22	0.93	0.36	136
Obfuscator.AD	0.65	1.00	0.79	142
Rbot!gen	0.93	0.99	0.96	158
Skintrim.N	0.00	0.00	0.00	80
Swizzor.gen!E	0.00	0.00	0.00	128
Swizzor.gen!I	0.00	0.00	0.00	132
VB.AT	0.00	0.00	0.00	408
Wintrim.BX	0.00	0.00	0.00	97
Yuner.A	0.00	0.00	0.00	800
Accuracy			0.76	
Macro Avg	0.57	0.73	0.61	9339
Weighted Avg	0.75	0.76	0.74	9339

Appendix A.3. Results: 50% Synthetic/50% Real

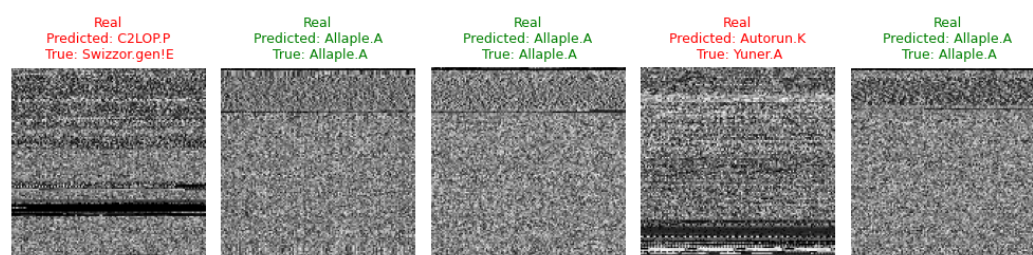
This experiment marks a significant improvement to 81% accuracy, with the model beginning to correctly identify families like Skintrim.N.

Table A3. Classification Report: 50% Synthetic/50% Real.

Family	Precision	Recall	F1-Score	Support
Adialer.C	0.99	1.00	1.00	122
Agent.FYI	0.88	1.00	0.94	116
Allaple.A	1.00	0.98	0.99	2949
Allaple.L	0.99	0.99	0.99	1591
Alueron.gen!J	0.98	0.93	0.95	198
Autorun.K	0.12	1.00	0.21	106
C2LOP.P	0.36	0.91	0.52	146
C2LOP.gen!g	0.42	0.84	0.56	200
Dialplatform.B	0.73	1.00	0.84	177
Dontovo.A	0.69	1.00	0.81	162
Fakerean	1.00	0.99	1.00	381
Instantaccess	0.99	0.98	0.98	431
Lolyda.AA1	0.97	0.94	0.95	213
Lolyda.AA2	0.98	0.97	0.98	184
Lolyda.AA3	0.98	0.99	0.99	123
Lolyda.AT	0.42	0.99	0.59	159

Table A3. *Cont.*

Family	Precision	Recall	F1-Score	Support
Malex.gen!J	0.87	0.93	0.90	136
Obfuscator.AD	0.93	1.00	0.96	142
Rbot!gen	0.85	1.00	0.92	158
Skintrim.N	0.99	1.00	0.99	80
Swizzor.gen!E	0.00	0.00	0.00	128
Swizzor.gen!I	0.00	0.00	0.00	132
VB.AT	0.00	0.00	0.00	408
Wintrim.BX	0.00	0.00	0.00	97
Yuner.A	0.00	0.00	0.00	800
Accuracy			0.81	
Macro Avg	0.65	0.78	0.68	9339
Weighted Avg	0.77	0.81	0.78	9339

**Figure A3.** Visualizing Model Predictions: 50% Synthetic/50% Real.*Appendix A.4. Results: 40% Synthetic/60% Real*

In this experiment, the accuracy reached 82%, and we see the first instances of correct classification for the Swizzor.gen!E family.

Table A4. Classification Report: 40% Synthetic/60% Real.

Family	Precision	Recall	F1-Score	Support
Adialer.C	0.97	1.00	0.98	122
Agent.FYI	0.99	0.97	0.98	116
Allapple.A	1.00	0.97	0.98	2949
Allapple.L	0.99	1.00	1.00	1591
Alueron.gen!J	0.99	0.96	0.98	198
Autorun.K	0.12	1.00	0.21	106
C2LOP.P	0.53	0.86	0.66	146
C2LOP.gen!g	0.62	0.85	0.72	200
Dialplatform.B	0.76	1.00	0.86	177
Dontovo.A	0.83	1.00	0.91	162
Fakerean	1.00	0.99	1.00	381
Instantaccess	1.00	1.00	1.00	431
Lolyda.AA1	0.96	1.00	0.98	213
Lolyda.AA2	0.88	0.97	0.92	184
Lolyda.AA3	0.98	0.99	0.98	123
Lolyda.AT	0.83	0.99	0.90	159
Malex.gen!J	0.71	0.93	0.80	136
Obfuscator.AD	1.00	1.00	1.00	142
Rbot!gen	0.97	0.99	0.98	158
Skintrim.N	1.00	1.00	1.00	80
Swizzor.gen!E	0.20	0.76	0.32	128
Swizzor.gen!I	0.00	0.00	0.00	132

Table A4. Cont.

Family	Precision	Recall	F1-Score	Support
VB.AT	0.00	0.00	0.00	408
Wintrim.BX	0.00	0.00	0.00	97
Yuner.A	0.00	0.00	0.00	800
Accuracy			0.82	
Macro Avg	0.69	0.81	0.73	9339
Weighted Avg	0.79	0.82	0.80	9339

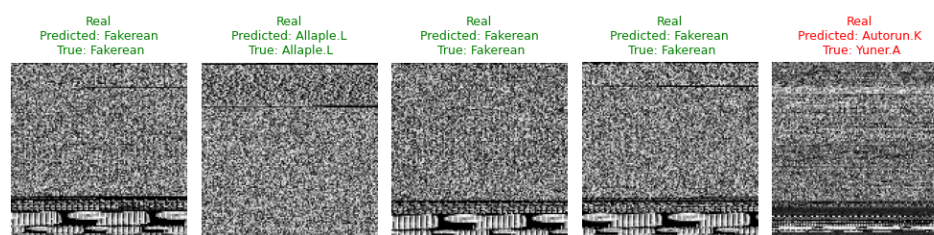


Figure A4. Visualizing Model Predictions: 40% Synthetic/60% Real.

Appendix A.5. Results: 30% Synthetic/70% Real

Accuracy remained at 82% in this experiment, but the model showed the first signs of correctly identifying the Swizzor.gen!I family.

Table A5. Classification Report: 30% Synthetic/70% Real.

Family	Precision	Recall	F1-Score	Support
Adialer.C	0.94	1.00	0.97	122
Agent.FYI	0.98	1.00	0.99	116
Allaple.A	1.00	0.98	0.99	2949
Allaple.L	1.00	1.00	1.00	1591
Alueron.gen!J	0.96	0.97	0.96	198
Autorun.K	0.12	1.00	0.21	106
C2LOP.P	0.54	0.79	0.64	146
C2LOP.gen!g	0.49	0.68	0.57	200
Dialplatform.B	0.98	0.99	0.99	177
Dontovo.A	0.59	1.00	0.74	162
Fakerean	1.00	0.99	1.00	381
Instantaccess	0.99	1.00	1.00	431
Lolyda.AA1	0.95	1.00	0.97	213
Lolyda.AA2	0.74	0.97	0.84	184
Lolyda.AA3	0.99	0.98	0.99	123
Lolyda.AT	0.99	0.99	0.99	159
Malex.gen!J	0.77	0.93	0.84	136
Obfuscator.AD	0.99	1.00	1.00	142
Rbot!gen	0.99	1.00	0.99	158
Skintrim.N	1.00	1.00	1.00	80
Swizzor.gen!E	0.20	0.67	0.31	128
Swizzor.gen!I	0.41	0.08	0.14	132
VB.AT	0.00	0.00	0.00	408
Wintrim.BX	0.00	0.00	0.00	97
Yuner.A	0.00	0.00	0.00	800
Accuracy			0.82	
Macro Avg	0.70	0.80	0.73	9339
Weighted Avg	0.79	0.82	0.80	9339

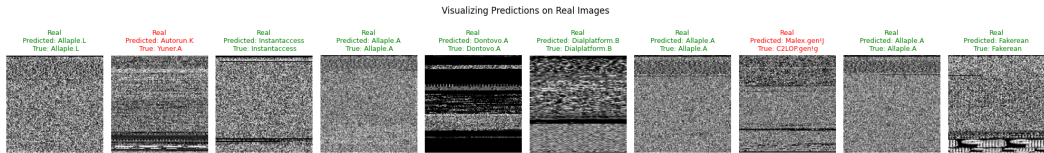


Figure A5. Visualizing Model Predictions: 30% Synthetic/70% Real.

Appendix A.6. Results: 20% Synthetic/80% Real

In this experiment, accuracy improved to 83%. The model increased its recall for both Swizzor variants, though overall accuracy was still inhibited by the VB.AT and Yuner.A families.

Table A6. Classification Report: 20% Synthetic/80% Real.

Family	Precision	Recall	F1-Score	Support
Adialer.C	0.98	1.00	0.99	122
Agent.FYI	0.85	1.00	0.92	116
Allapple.A	1.00	0.98	0.99	2949
Allapple.L	0.99	1.00	1.00	1591
Alueron.gen!J	1.00	0.99	1.00	198
Autorun.K	0.12	1.00	0.21	106
C2LOP.P	0.57	0.85	0.68	146
C2LOP.gen!g	0.57	0.79	0.67	200
Dialplatform.B	0.95	1.00	0.98	177
Dontovo.A	0.51	1.00	0.68	162
Fakerean	1.00	1.00	1.00	381
Instantaccess	1.00	1.00	1.00	431
Lolyda.AA1	0.96	1.00	0.98	213
Lolyda.AA2	0.83	0.97	0.90	184
Lolyda.AA3	0.94	0.99	0.96	123
Lolyda.AT	0.95	0.99	0.97	159
Malex.gen!J	0.91	0.93	0.92	136
Obfuscator.AD	0.99	1.00	1.00	142
Rbot!gen	0.98	0.99	0.99	158
Skintrim.N	1.00	1.00	1.00	80
Swizzor.gen!E	0.20	0.49	0.29	128
Swizzor.gen!I	0.42	0.34	0.38	132
VB.AT	0.00	0.00	0.00	408
Wintrim.BX	0.00	0.00	0.00	97
Yuner.A	0.00	0.00	0.00	800
Accuracy			0.83	
Macro Avg	0.71	0.81	0.74	9339
Weighted Avg	0.80	0.83	0.80	9339

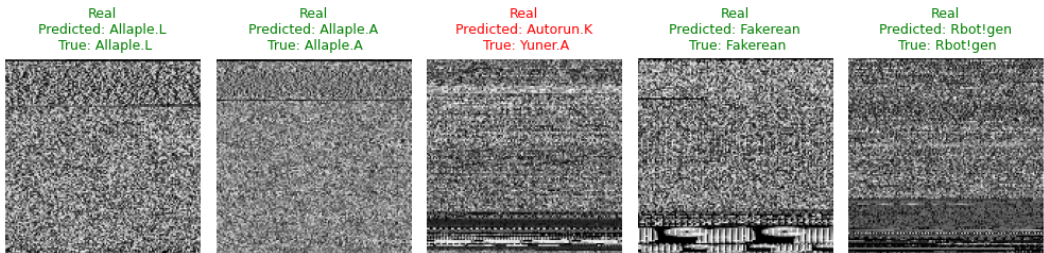


Figure A6. Visualizing Model Predictions: 20% Synthetic/80% Real.

Appendix A.7. Results: 10% Synthetic/90% Real

In this experiment, the accuracy increased to 85%. Crucially, the model finally began to correctly classify members of the VB.AT family as the training set became 90% authentic.

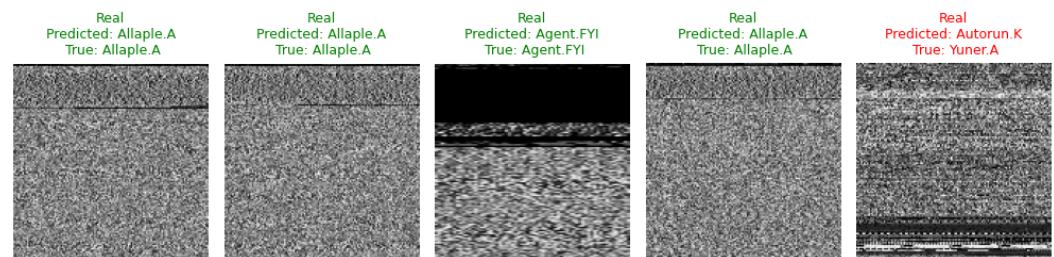


Figure A7. Visualizing Model Predictions: 10% Synthetic/90% Real.

Table A7. Classification Report: 10% Synthetic/90% Real.

Family	Precision	Recall	F1-Score	Support
Adialer.C	1.00	1.00	1.00	122
Agent.FYI	0.99	0.97	0.98	116
Allapple.A	1.00	0.98	0.99	2949
Allapple.L	0.99	0.99	0.99	1591
Alueron.gen!J	1.00	0.92	0.96	198
Autorun.K	0.12	1.00	0.21	106
C2LOP.P	0.52	0.85	0.65	146
C2LOP.gen!g	0.49	0.71	0.58	200
Dialplatform.B	0.96	1.00	0.98	177
Dontovo.A	0.87	1.00	0.93	162
Fakerean	1.00	0.99	1.00	381
Instantaccess	0.98	1.00	0.99	431
Lolyda.AA1	0.92	1.00	0.96	213
Lolyda.AA2	1.00	0.97	0.99	184
Lolyda.AA3	0.98	0.98	0.98	123
Lolyda.AT	0.76	1.00	0.86	159
Malex.gen!J	0.95	0.93	0.94	136
Obfuscator.AD	0.99	1.00	0.99	142
Rbot!gen	0.96	1.00	0.98	158
Skintrim.N	1.00	1.00	1.00	80
Swizzor.gen!E	0.34	0.54	0.42	128
Swizzor.gen!I	0.41	0.32	0.36	132
VB.AT	1.00	0.62	0.76	408
Wintrim.BX	0.00	0.00	0.00	97
Yuner.A	0.00	0.00	0.00	800
Accuracy			0.85	
Macro Avg	0.77	0.83	0.78	9339
Weighted Avg	0.85	0.85	0.84	9339

Appendix A.8. Class-Specific Attenuation and Boundary Breaking Points

A granular analysis of the sequential classification performance reports across varying synthetic-to-real ratios (Table A1 through Table A7) reveals structural behavioral trends that macro-level accuracy metrics obscure. While the global accuracy scales monotonically from 76.0% to 85.0% as the training configuration shifts from 70% Synthetic/30% Real to 10% Synthetic/90% Real, this performance growth is driven by sudden, non-linear optimization breakthroughs in specific malware families rather than uniform improvement across the dataset.

The most prominent architectural bottleneck observed is the total systemic blindness to the *Yuner.A* family, which maintains an F1-score of exactly 0.00 across all evaluated ratios.

Despite comprising nearly 9% of the testing distribution (*Support* = 800), the features learned from the synthetic generation pipeline fail to align with the empirical reality of the *Yuner.A* target domain, causing a persistent mathematical depression of the upper-bound global accuracy. A similar phenomenon is present in *Autorun.K* (*Support* = 106), which exhibits a static recall of 1.00 alongside a severely depressed precision of 0.12 across all experiments. This reveals a highly warped feature space where the model routinely utilizes *Autorun.K* as a majority-class sink, misclassifying unmapped features from zero-F1 families into this specific boundary zone.

Conversely, the transition to high-proportion authentic training regimes exposes clear baseline threshold dependencies for complex classes. The *VB.AT* family (*Support* = 408) remains completely unclassified (F1-score of 0.00) from the 70% synthetic configuration down through the 20% synthetic configuration. However, upon reaching the 10% Synthetic/90% Real training split, *VB.AT* undergoes an abrupt optimization breakthrough, achieving an F1-score of 0.76 (*Precision* = 1.00, *Recall* = 0.62). This indicates that certain malware families exhibit a strict floor of authenticity requirements, that is, their internal structural variance cannot be adequately bridged by the generative distribution, requiring a dominant presence of real-world data to converge during backpropagation.

References

1. McAfee Labs. *Global Threat Landscape Report: The Proliferation of Polymorphic Malware*; Technical Report; McAfee Enterprise: San Jose, CA, USA, 2025.
2. SonicWall Cyber Threat Research. *2026 SonicWall Cyber Threat Report: Persistent Evasions and Evolving Attack Vectors*; Technical Report; SonicWall, Inc.: Milpitas, CA, USA, 2026.
3. Vi, B.N.; Noi Nguyen, H.; Nguyen, N.T.; Truong Tran, C. Adversarial Examples Against Image-based Malware Classification Systems. In Proceedings of the 2019 11th International Conference on Knowledge and Systems Engineering (KSE), Da Nang, Vietnam, 24–26 October 2019; pp. 1–5. [\[CrossRef\]](#)
4. Alshoulie, M.; Mehmood, A. Deep Learning Approaches for Malware Detection: A Comprehensive Review of Techniques, Challenges, and Future Directions. *IEEE Access* **2025**, *13*, 118652–118677. [\[CrossRef\]](#)
5. Ayccock, J. Anti-Virus Techniques. In *Computer Viruses and Malware*; Springer: Boston, MA, USA, 2006; pp. 53–95. [\[CrossRef\]](#) [\[PubMed\]](#)
6. Agarap, A.F. Towards building an intelligent anti-malware system: A deep learning approach using support vector machine (SVM) for malware classification. *arXiv* **2017**, arXiv:1801.00318.
7. Kalash, M.; Rochan, M.; Mohammed, N.; Bruce, N.D.B.; Wang, Y.; Iqbal, F. Malware Classification with Deep Convolutional Neural Networks. In Proceedings of the 2018 9th IFIP International Conference on New Technologies, Mobility and Security (NTMS), Paris, France, 26–28 February 2018; pp. 1–5. [\[CrossRef\]](#)
8. Gnaneswar, S.S.; N, A.; M, R.; Bopche, G.S. From Bytes to Pixels: Robust Malware Classification Using Deep Neural Networks. In Proceedings of the 2025 1st International Conference on Data Science and Intelligent Network Computing (ICDSINC), Raipur, India, 9–11 December 2025; pp. 823–829. [\[CrossRef\]](#)
9. Nataraj, L.; Karthikeyan, S.; Jacob, G.; Manjunath, B.S. Malware Images: Visualization and Automatic Classification. In Proceedings of the 8th International Symposium on Visualization for Cyber Security (VizSec), Pittsburgh, PA, USA, 20 July 2011; pp. 1–7. [\[CrossRef\]](#)
10. Chandran, S.; Syam, S.R.; Sankaran, S.; Pandey, T.; Achuthan, K. From Static to AI-Driven Detection: A Comprehensive Review of Obfuscated Malware Techniques. *IEEE Access* **2025**, *13*, 74335–74358. [\[CrossRef\]](#)
11. Van den Oord, A.; Kalchbrenner, N.; Kavukcuoglu, K. Pixel Recurrent Neural Networks. In Proceedings of the 33rd International Conference on Machine Learning, New York, NY, USA, 19–24 June 2016; Volume 48, pp. 1747–1756.
12. Peng, D.; Husain, M.; Siddiqui, A.; Bhattacharya, S. Visual Malware Classification Using a CNN. In Proceedings of the 2024 IEEE MIT Undergraduate Research Technology Conference (URTC), Cambridge, MA, USA, 11–13 October 2024; pp. 1–5. [\[CrossRef\]](#)
13. Yajamanam, S.; Selvin, V.R.S.; Di Troia, F.; Stamp, M. Deep Learning versus Gist Descriptors for Image-based Malware Classification. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy-Volume 1: ForSE*; INSTICC; SciTePress: Setúbal, Portugal, 2018; pp. 553–561. [\[CrossRef\]](#)
14. Vasan, D.; Alazab, M.; Wassan, S.; Naeem, H.; Safaei, B.; Zheng, Q. IMCFN: Image-based malware classification using fine-tuned convolutional neural network architecture. *Comput. Netw.* **2020**, *171*, 107138. [\[CrossRef\]](#)

15. Nguyen, H.; Di Troia, F.; Ishigaki, G.; Stamp, M. Generative adversarial networks and image-based malware classification. *J. Comput. Virol. Hacking Tech.* **2023**, *19*, 579–595. [[CrossRef](#)]
16. Hu, W.; Tan, Y. Generating adversarial malware examples for black-box attacks based on GAN. In *Proceedings of the International Conference on Data Mining and Big Data*; Springer: Berlin/Heidelberg, Germany, 2022; pp. 409–423.
17. Trehan, H.; Di Troia, F. Fake Malware Generation Using HMM and GAN. In *Proceedings of the Silicon Valley Cybersecurity Conference*; Chang, S.Y., Bathen, L., Di Troia, F., Austin, T.H., Nelson, A.J., Eds.; Springer: Cham, Switzerland, 2022; pp. 3–21.
18. Mazaed Alotaibi, F.; Fawad. A Multifaceted Deep Generative Adversarial Networks Model for Mobile Malware Detection. *Appl. Sci.* **2022**, *12*, 9403. [[CrossRef](#)]
19. Ho, J.; Jain, A.; Abbeel, P. Denoising diffusion probabilistic models. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 6840–6851.
20. Bao, T.; Trousil, K.; Duy Tran, Q.; Di Troia, F.; Park, Y. Generating Synthetic Malware Samples Using Generative AI. *IEEE Access* **2025**, *13*, 59725–59736. [[CrossRef](#)]
21. Sun, Y.; Ji, Y.; Tao, X. Research on Default Classification of Unbalanced Credit Data Based on PixelCNN-WGAN. *Electronics* **2024**, *13*, 3419. [[CrossRef](#)]
22. Van den Oord, A.; Vinyals, O.; Kavukcuoglu, K. Neural Discrete Representation Learning. In *Proceedings of the Advances in Neural Information Processing Systems (NeurIPS)*; Curran Associates, Inc.: Red Hook, NY, USA, 2017; Volume 30.
23. Parmar, N.; Vaswani, A.; Uszkoreit, J.; Kaiser, L.; Shazeer, N.; Ku, A.; Tran, D. Image Transformer. In *Proceedings of the International Conference on Machine Learning (ICML)*, Stockholm, Sweden, 10–15 July 2018; pp. 4055–4064.
24. Esser, P.; Rombach, R.; Ommer, B. Taming Transformers for High-Resolution Image Synthesis. In *Proceedings of the Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Nashville, TN, USA, 20–25 June 2021; pp. 12873–12883.
25. Ravuri, S.; Vinyals, O. Classification Accuracy Score for Conditional Generative Models. In *Proceedings of the Advances in Neural Information Processing Systems (NeurIPS)*; Curran Associates, Inc.: Red Hook, NY, USA, 2019; Volume 32.
26. UCSB Vision and Robotics Group. Signal Processing for Malware Analysis. Available online: <https://vision.ece.ucsb.edu/research/signal-processing-malware-analysis> (accessed on 26 June 2026).
27. Van den Oord, A.; Kalchbrenner, N.; Espeholt, L.; Vinyals, O.; Graves, A.; Kavukcuoglu, K. Conditional Image Generation with PixelCNN Decoders. In *Proceedings of the Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2016; Volume 29, pp. 4790–4798.
28. Salimans, T.; Karpathy, A.; Chen, X.; Kingma, D.P. PixelCNN++: Improving PixelCNN with Discretized Logistic Mixture Likelihood and Other Modifications. In *Proceedings of the International Conference on Learning Representations (ICLR)*, Toulon, France, 24–26 April 2017.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.